Week 3 - cfn-init, cfn-signal, cfn-hup, CreationPolicy, WaitCondition, UpdatePolicy (ASG rolling / replacing)

DOP-C02 Domain 2 (Configuration Management and IaC). CloudFormation creates an EC2 instance in seconds but **doesn't know whether the software works**. These features fix that:

- **Configure** declaratively: `cfn-init` with `AWS::CloudFormation::Init`.
- **Report** success or failure: `cfn-signal`.
- **Wait** for the report: `CreationPolicy` or `WaitCondition`.
- **Apply metadata changes** to running instances: `cfn-hup`.
- **Roll out changes** to Auto Scaling groups safely: `UpdatePolicy` (rolling or replacing).

Part 1 - Why not just UserData?

	Plain UserData script	cfn-init + metadata
Style	Imperative bash, top to bottom	**Declarative**: packages, files, services in the template
Runs	**Only at first boot**	First boot, and **again on metadata change** with cfn-hup
Reports success to CloudFormation	No: stack complete once the instance runs	**Yes**, with cfn-signal + CreationPolicy
Updating config later	Changing UserData needs a **stop/start** and the script **won't rerun**	**Change the metadata**; cfn-hup applies it **in place**
Readability	Long scripts are hard to review	Structured, shown in change sets

- **Exam point:** stack shows CREATE_COMPLETE but the app isn't installed or failed: use **cfn-signal with a CreationPolicy**.
- **Helper scripts** (`aws-cfn-bootstrap`): **preinstalled on Amazon Linux** in `/opt/aws/bin/`. On **Ubuntu, RHEL, Windows**, **install** them first in UserData, otherwise commands aren't found and the stack times out.
- Scripts: `cfn-init`, `cfn-signal`, `cfn-hup`, `cfn-get-metadata`.

Part 2 - AWS::CloudFormation::Init (the metadata cfn-init reads)

```
WebServer:
  Type: AWS::EC2::Instance
  Metadata:
    AWS::CloudFormation::Init:
      configSets:
        default: [install, configure]      # run "install" first, then "configure"
      install:
        packages:
          yum:
            httpd: []
            php: []
        groups:
          webadmins: {}
        users:
          deploy: { groups: [webadmins], homeDir: /home/deploy }
        sources:
          /var/www/html: https://my-bucket.s3.amazonaws.com/site-1.4.zip   # download + extract
      configure:
        files:
          /var/www/html/config.php:
            content: !Sub |
              <?php $env = "${Environment}"; $db = "${DbEndpoint}"; ?>
            mode: "000640"
            owner: apache
            group: apache
          /etc/cfn/cfn-hup.conf:                       # cfn-hup settings (Part 6)
            content: !Sub |
              [main]
              stack=${AWS::StackId}
              region=${AWS::Region}
              interval=5
            mode: "000400"
          /etc/cfn/hooks.d/cfn-auto-reloader.conf:
            content: !Sub |
              [cfn-auto-reloader-hook]
              triggers=post.update
              path=Resources.WebServer.Metadata.AWS::CloudFormation::Init
              action=/opt/aws/bin/cfn-init -v --stack ${AWS::StackName} --resource WebServer --configsets
    default --region ${AWS::Region}
              runas=root
        commands:
          01_migrate:
            command: php /var/www/html/migrate.php
            cwd: /var/www/html
            test: test -f /var/www/html/migrate.php     # run only if the test passes
            ignoreErrors: false
        services:
          sysvinit:
            httpd:
              enabled: true
              ensureRunning: true
              files: [/var/www/html/config.php]        # restart httpd if this file changes
            cfn-hup:
              enabled: true
              ensureRunning: true
              files: [/etc/cfn/cfn-hup.conf, /etc/cfn/hooks.d/cfn-auto-reloader.conf]
```

Config keys and execution order

Within one config, keys always run in this order, **whatever order you write them**:

packages, groups, users, sources, files, commands, services

Key	Notes
`packages`	`yum`, `apt`, `rpm`, `python` (pip), `rubygems`, `msi`. Empty list = latest, or pin versions
`groups` / `users`	Linux groups and users
`sources`	Download an archive (zip, tar, tgz) from a URL or S3 and **extract** it
`files`	Inline `content` (works with `!Sub`) or a `source` URL; `mode`, `owner`, `group`

Key	Notes
`commands`	**Alphabetical order by name** (`01_`, `02_`); `test`, `cwd`, `env`, `ignoreErrors`
`services`	`sysvinit` or `windows`: `enabled`, `ensureRunning`, **restart when listed** `files`, `sources`, `packages` **or** `commands` **change**

- **configSets** choose **which configs run and in what order** (`default: [install, configure]`), can include other configSets, and are selected with `cfn-init --configsets <name>`. Without configSets, a single config named `config` runs.
- **Private S3 files:** `AWS::CloudFormation::Authentication` with `type: S3` and `roleName` (the **instance profile role**) or `buckets`.

Part 3 - cfn-init

Reads the metadata from CloudFormation and **applies** it. Called from UserData:

```
UserData:
  Fn::Base64: !Sub |
    #!/bin/bash -xe
    yum update -y aws-cfn-bootstrap
    /opt/aws/bin/cfn-init -v --stack ${AWS::StackName} --resource WebServer \
        --configsets default --region ${AWS::Region}
    /opt/aws/bin/cfn-signal -e $? --stack ${AWS::StackName} --resource WebServer --region ${AWS::Region}
```

- **Arguments:** `--stack`, `--resource` (logical ID of the resource **holding the metadata**; for an ASG usually the **launch template**), `--configsets`, `--region`, `-v`, `--role`.
- **Logs:** `/var/log/cfn-init.log`, `/var/log/cfn-init-cmd.log`, `/var/log/cloud-init-output.log`; Windows `C:\cfn\log\`.
- **Network:** the instance must reach the **CloudFormation endpoint** (and S3 for downloads). **Private subnets** need a **NAT gateway** or **VPC interface endpoints for CloudFormation** (`com.amazonaws.<region>.cloudformation`) plus S3. Otherwise nothing signals and the stack **times out**.
- **Exit code:** non-zero if any step fails; pass it to `cfn-signal -e $?`.

Part 4 - cfn-signal and CreationPolicy

cfn-signal

```
/opt/aws/bin/cfn-signal -e $? --stack ${AWS::StackName} --resource WebServerGroup --region ${AWS::Region}
# or explicit:      --success true|false
# optional:         --reason "text"   -i <unique id>   -d "data"  (for wait conditions)
```

- `-e $?` turns the previous exit code into success (0) or failure.
- Signal **after verifying the app** (for example `curl -f http://localhost/health`), not just after installing.
- `--resource` = the resource with the **CreationPolicy**: the **ASG** for groups, the **instance** for a single instance.

CreationPolicy

```
WebServerGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  CreationPolicy:
    ResourceSignal:
      Count: 3                # number of SUCCESS signals needed
      Timeout: PT15M          # ISO 8601 duration, maximum PT12H (12 hours)
    AutoScalingCreationPolicy:
      MinSuccessfulInstancesPercent: 100   # % of instances that must signal success
  Properties:
    MinSize: "3"
    MaxSize: "6"
    DesiredCapacity: "3"
    LaunchTemplate:
      LaunchTemplateId: !Ref WebLaunchTemplate
      Version: !GetAtt WebLaunchTemplate.LatestVersionNumber
    VPCZoneIdentifier: !Ref Subnets
```

- **Supported on** `AWS::EC2::Instance`, `AWS::AutoScaling::AutoScalingGroup`, `AWS::CloudFormation::WaitCondition` (and a few others such as AppStream fleets).
- The resource stays **CREATE_IN_PROGRESS** until `Count` success signals arrive. A **FAILURE** signal or the `Timeout` fails it, and the stack **rolls back**.
- **Timeout:** ISO 8601 (`PT5M`, `PT1H`), **maximum 12 hours**.
- **Messages:** "Failed to receive 3 resource signal(s) within the specified duration"; "Received 1 FAILURE signal(s) out of 3. Unable to satisfy 100% MinSuccessfulInstancesPercent requirement".
- **Dependent resources wait too** (for example a DNS record that `DependsOn` the ASG).

Part 5 - WaitCondition and WaitConditionHandle

```
AppReadyHandle:
  Type: AWS::CloudFormation::WaitConditionHandle      # produces a presigned S3 URL

AppReady:
  Type: AWS::CloudFormation::WaitCondition
  DependsOn: AppServer                                  # start waiting after the server exists
  Properties:
    Handle: !Ref AppReadyHandle
    Count: 1
    Timeout: "1800"                                     # seconds, maximum 43200 (12 hours)

AppServer:
  Type: AWS::EC2::Instance
  Properties:
    UserData:
      Fn::Base64: !Sub |
        #!/bin/bash
        ... install ...
        /opt/aws/bin/cfn-signal -e $? -d "version=1.4" '${AppReadyHandle}'

Outputs:
  SignalData:
    Value: !GetAtt AppReady.Data        # JSON of the data sent with signals
```

- **The handle is a presigned S3 URL**: anything that can **HTTP PUT** can signal (scripts, on-premises servers, external systems, Lambda). Body example: `{"Status":"SUCCESS","Reason":"done","UniqueId":"1","Data":"version=1.4"}`.
- `Fn::GetAtt WaitCondition.Data` returns the data sent with signals.
- Private subnets need a route to **S3** for the signal.
- **Use WaitCondition when:** coordinating with **external systems** or resources **outside** the stack; waiting at a specific point after another resource (`DependsOn`); **receiving data** back.
- **For EC2 and ASG, AWS recommends CreationPolicy.** A WaitCondition can also use a CreationPolicy itself, without a handle.
- **Updates:** a wait condition **doesn't wait again**; use a **new handle** (new logical ID) to wait again.

Part 6 - cfn-hup (apply metadata changes to running instances)

`cfn-hup` is a **daemon** that **polls the resource metadata** and runs **hook actions** on change, usually **rerunning cfn-init**.

File	Purpose
`/etc/cfn/cfn-hup.conf`	`[main]` with `stack=`, `region=`, `interval=` (minutes, **default 15**)
`/etc/cfn/hooks.d/*.conf`	`triggers=post.update`, `path=Resources.<LogicalId>.Metadata.AWS::CloudFormation::Init`, `action=` (usually cfn-init), `runas=root`

- Create both with cfn-init `files`, keep cfn-hup running with `services` (Part 2 example).
- **Result:** change a file's content in the **metadata**, update the stack, and within the interval cfn-hup reruns cfn-init, rewrites the file and restarts the service. **No replacement**; change set shows **Metadata** changes.
- **Without cfn-hup**, metadata-only changes update the stack but **running instances don't change**.
- **Modern alternatives: SSM State Manager / Run Command**, or **golden AMIs** (Image Builder) with **instance replacement**.

Part 7 - UpdatePolicy for Auto Scaling groups

An `UpdatePolicy` controls **how CloudFormation updates an ASG** when the change affects instances (new **launch template version**, instance type, UserData, subnets). **Without an UpdatePolicy**, changing the launch template **doesn't replace existing instances**; only new instances use it.

1. AutoScalingRollingUpdate - batches inside the same ASG

```
WebServerGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  UpdatePolicy:
    AutoScalingRollingUpdate:
      MaxBatchSize: 2                    # instances replaced per batch
      MinInstancesInService: 4           # keep at least this many in service
      MinSuccessfulInstancesPercent: 100 # % of new instances that must signal success
      PauseTime: PT10M                   # wait between batches / for signals (ISO 8601)
      WaitOnResourceSignals: true        # wait for cfn-signal from each new instance
      SuspendProcesses:                  # stop ASG from interfering during the update
        - HealthCheck
        - ReplaceUnhealthy
        - AZRebalance
        - AlarmNotification
        - ScheduledActions
```

Property	Meaning
`MaxBatchSize`	Instances replaced at once
`MinInstancesInService`	Capacity floor during the update
`MinSuccessfulInstancesPercent`	Success threshold; below it the update fails and **rolls back**
`PauseTime`	Wait after each batch; with `WaitOnResourceSignals: true` it is the **signal timeout**
`WaitOnResourceSignals`	Wait for **cfn-signal** from new instances before the next batch
`SuspendProcesses`	Suspend ASG processes so scaling, health replacement or AZ rebalancing don't interfere

- Old instances are replaced batch by batch **in the same ASG**. Rollback = **another rolling update** (slow).

2. AutoScalingReplacingUpdate - a whole new ASG (immutable)

```
WebServerGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  UpdatePolicy:
    AutoScalingReplacingUpdate:
      WillReplace: true
  CreationPolicy:                    # judges the NEW group
    ResourceSignal: { Count: 3, Timeout: PT15M }
    AutoScalingCreationPolicy: { MinSuccessfulInstancesPercent: 100 }
```

- CloudFormation **creates a new ASG**, waits for its **CreationPolicy signals**, then **deletes the old ASG** during cleanup.
- On failure it **deletes the new ASG**; the old group was **never touched**: **fast, safe rollback**.
- Costs: briefly **double capacity**; the new ASG gets a **new physical name**.
- If both are set, **WillReplace wins**.

3. AutoScalingScheduledAction

```
UpdatePolicy:
  AutoScalingScheduledAction:
    IgnoreUnmodifiedGroupSizeProperties: true
```

- Stops stack updates from **resetting** `MinSize`, `MaxSize`, `DesiredCapacity` changed by **scheduled actions**, when the template didn't change them.

Rolling vs replacing

	AutoScalingRollingUpdate	AutoScalingReplacingUpdate
How	Batches in the **same ASG**	**New ASG**, then old one deleted
Capacity	Controlled (`MinInstancesInService`)	Briefly **double**
Rollback	Another rolling update (slow)	**Delete the new ASG** (fast, old untouched)
Success check	`WaitOnResourceSignals` + `MinSuccessfulInstancesPercent`	**CreationPolicy** on the new group
Similar to	Rolling deployment	**Immutable** / blue-green style
Exam keyword	Keep N in service, update in batches	Fastest rollback, don't touch existing instances, immutable

Other UpdatePolicy types

- `CodeDeployLambdaAliasUpdate`**:** Lambda alias traffic shifting through CodeDeploy.
- `UseOnlineResharding`**:** ElastiCache shards without replacement.
- `EnableVersionUpgrade`**:** OpenSearch / Elasticsearch version upgrade **in place**.

Auto Scaling traps

- **Don't hard-code** `DesiredCapacity` if scaling policies change it: every update **resets** it.
- **ASG instance refresh** is the ASG's own rolling replacement; in CloudFormation, the **UpdatePolicy** controls replacement during stack updates.

Part 8 - Putting it together

```
LaunchTemplate  (Metadata: AWS::CloudFormation::Init; UserData: cfn-init + health check + cfn-signal)
      |
      v
AutoScalingGroup
   CreationPolicy: ResourceSignal Count = desired, Timeout PT15M     -> create waits for healthy instances
   UpdatePolicy:   AutoScalingRollingUpdate (WaitOnResourceSignals)  -> safe batches
                   or AutoScalingReplacingUpdate (WillReplace)       -> immutable updates
      |
cfn-hup on each instance  -> metadata-only changes applied in place
```

- For an ASG: **metadata on the launch template** (`cfn-init --resource WebLaunchTemplate`), **signal to the ASG** (`cfn-signal --resource WebServerGroup`).

Part 9 - Troubleshooting: stack timed out waiting for signals

Cause	Fix
cfn-signal never ran (script failed earlier with `-e`, or no signal line)	Always signal, with `-e $?` or a trap / error handler; check `cloud-init-output.log`
Wrong `--resource` (for example the launch template instead of the ASG), wrong stack or Region	Signal the resource **with the CreationPolicy**
No network path to CloudFormation (or S3 for wait conditions)	**NAT gateway** or **CloudFormation VPC endpoint** (+ S3 endpoint)
aws-cfn-bootstrap not installed (Ubuntu, RHEL, Windows)	Install it in UserData first
UserData didn't run (no `#!/bin/bash`, not Base64, quoting error)	`Fn::Base64` and a proper shebang; check cloud-init logs
Timeout too short	Increase `Timeout` / `PauseTime`, or bake software into an AMI
App installed but health check failed (FAILURE sent)	`/var/log/cfn-init.log`, `cfn-init-cmd.log`, app logs
Can't see the instance after rollback	**Disable rollback** (`--disable-rollback` or `--on-failure DO_NOTHING`, "preserve successfully provisioned resources"), inspect, then delete
ASG `Count` larger than `DesiredCapacity`	Match `Count` to the instances that will signal

Part 10 - Exam traps

Trap	Truth
Stack CREATE_COMPLETE before the app is ready	**CreationPolicy** + **cfn-signal**
Changed UserData, so running instances get the new script	UserData runs **at first boot only**. Use **cfn-hup + metadata** or replace instances
Changed metadata but the instance didn't change	Needs **cfn-hup** with a `post.update` hook rerunning cfn-init
cfn-init runs commands before files	Order: **packages, groups, users, sources, files, commands, services**; commands alphabetical
Restart Apache when its config file changes	`services`, `sysvinit`, `httpd`, `files: [config]`
Download a private S3 artifact with cfn-init	`AWS::CloudFormation::Authentication` (S3, roleName) + instance profile
Signal the launch template resource	Signal the resource **with the CreationPolicy** (ASG or instance)
Timeout of 24 hours	Maximum **12 hours** (`PT12H` / 43200 seconds)
On-premises system must confirm before the stack continues	**WaitCondition + WaitConditionHandle** (presigned URL, HTTP PUT)
Pass a value from the instance into the stack	WaitCondition with `-d` + `GetAtt WaitCondition.Data`
WaitCondition waits again on every update	**No**; new handle (new logical ID)
Private subnet instances time out signalling	**NAT** or **CloudFormation VPC endpoint** (+ S3 endpoint)
Ubuntu instance: cfn-init not found	Install **aws-cfn-bootstrap**
Fastest, safest ASG rollback; don't touch existing instances	**AutoScalingReplacingUpdate (WillReplace: true)** + CreationPolicy
Update in batches keeping 4 in service	**AutoScalingRollingUpdate** `MinInstancesInService: 4`, `MaxBatchSize`
Rolling update must wait until each new instance is healthy	`WaitOnResourceSignals: true` + `PauseTime` timeout + cfn-signal
A scaling event replaced instances during the rolling update	`SuspendProcesses`
Stack update resets scheduled capacity	`IgnoreUnmodifiedGroupSizeProperties: true`; avoid hard-coded DesiredCapacity
Both rolling and replacing UpdatePolicy set	**WillReplace takes precedence**
Changed launch template but old instances keep the old AMI	**No UpdatePolicy**: add rolling or replacing (or instance refresh)
Need to inspect a failed instance but rollback deleted it	Disable rollback / preserve resources, inspect, clean up

Part 11 - Hands-on lab (about 90 minutes, low cost; t3.micro, public subnet)

Lab 1 - cfn-init + cfn-signal + CreationPolicy on one instance

1. `web.yaml` with an `AWS::EC2::Instance` (latest Amazon Linux SSM parameter), a security group with port 80, and:
   - **Metadata** `AWS::CloudFormation::Init`**:** `packages: yum: httpd`; file `/var/www/html/index.html` with `!Sub "<h1>${AWS::StackName} v1</h1>"`; `services: sysvinit: httpd` enabled / ensureRunning, `files` listing index.html.
   - **UserData:** `cfn-init -v --stack ${AWS::StackName} --resource Web --region ${AWS::Region}`, then `curl -f http://localhost/`, then `cfn-signal -e $? --stack ${AWS::StackName} --resource Web --region ${AWS::Region}`.
   - **CreationPolicy:** `ResourceSignal: { Count: 1, Timeout: PT10M }`.
2. Deploy: the instance stays **CREATE_IN_PROGRESS** until the signal, then completes. Open the page.
3. Connect (Session Manager) and read `/var/log/cfn-init.log` and `/var/log/cfn-init-cmd.log`.

Lab 2 - Timeout and failure

1. Change cfn-signal to `--resource Wrong`, new stack with `Timeout: PT5M`: fails with "Failed to receive 1 resource signal(s)" and rolls back.
2. Fix it; add a bad package (`yum: notapackage`): cfn-init fails, **FAILURE signal** sent, stack fails fast.
3. Create again with `--disable-rollback`, connect to the instance and find the error in the log.

Lab 3 - cfn-hup applies metadata changes in place

1. Add the cfn-hup files (Part 2, `interval=1`) and the `cfn-hup` service.
2. Deploy, change index.html to `v2` in the **metadata only**, update: change set shows **Modify, Scope: Metadata, no replacement**.
3. Within about a minute the page shows **v2**, same instance ID.
4. Compare: change only **UserData** and update: instance stop/start, page **doesn't** change.

Lab 4 - ASG with rolling and replacing updates

1. Convert to a **launch template** (with metadata) + **ASG** (`MinSize: 2`, `DesiredCapacity: 2`), **CreationPolicy** `Count: 2`, cfn-signal `--resource WebGroup`, cfn-init `--resource WebLaunchTemplate`.
2. Add `AutoScalingRollingUpdate: { MaxBatchSize: 1, MinInstancesInService: 1, WaitOnResourceSignals: true, PauseTime: PT10M, SuspendProcesses: [HealthCheck, ReplaceUnhealthy, AZRebalance, AlarmNotification, ScheduledActions] }`.
3. Change the instance type and update: instances replaced **one at a time**, each waiting for its signal.
4. Switch to `AutoScalingReplacingUpdate: { WillReplace: true }`, change something, update: a **new ASG** appears, then the old one is deleted.
5. Break UserData (`exit 1` before the signal) and update: the new ASG fails and is deleted; **the old ASG keeps serving**.

Lab 5 (optional) - WaitCondition with data

1. Add a handle + WaitCondition (`Count 1`, `Timeout 900`); UserData `cfn-signal -d "built=$(date +%s)" '${Handle}'`.
2. Output `!GetAtt AppReady.Data` and see the JSON.

Clean up

- Delete all lab stacks.

Part 12 - Check yourself

Questions

1. Why might a stack show CREATE_COMPLETE although the app failed to install? Fix?
2. In what order does cfn-init process config keys?
3. How do you restart Apache automatically when cfn-init changes its config file?
4. A template updates only instance metadata, but the running instance doesn't change. What is missing?
5. Why doesn't changing UserData update running instances?
6. In an ASG setup, which resource does `cfn-signal --resource` name, and which holds the metadata for `cfn-init --resource`?
7. Maximum CreationPolicy timeout and format?
8. Private-subnet instances without NAT always time out signalling. Why, and fixes?
9. When use a WaitCondition instead of a CreationPolicy? Two cases.
10. How do you return a value from an instance into stack outputs?
11. Which UpdatePolicy gives the fastest rollback and never touches existing instances? What else is required?
12. During a rolling update a health-check replacement confused the update. Which setting prevents it?
13. Scheduled actions set desired capacity to 10, and a stack update reset it to 2. How do you stop this?
14. The launch template has a new AMI but existing instances still run the old one after the update. Why?
15. Both `AutoScalingReplacingUpdate` (WillReplace true) and `AutoScalingRollingUpdate` are defined. Which applies?

Answers

1. CloudFormation only knows the instance is **running**. Add a **CreationPolicy** and **cfn-signal** (`-e $?` after cfn-init and a health check).
2. **packages, groups, users, sources, files, commands, services**; commands alphabetical.
3. `services`, `sysvinit`, `httpd`, list the file under `files:`.
4. **cfn-hup** with a `post.update` hook (`path=...Metadata.AWS::CloudFormation::Init`, `action=cfn-init ...`) and the service running.
5. UserData runs **only at first boot**; a change needs a stop/start and still won't rerun.
6. **cfn-signal: the ASG** (resource with the CreationPolicy / UpdatePolicy). **cfn-init: the launch template** holding the metadata.
7. **12 hours**, ISO 8601 (`PT15M`, `PT12H`). WaitCondition `Timeout` in seconds, max 43200.
8. Helper scripts must reach the **CloudFormation endpoint** (and S3). Add **NAT**, or **CloudFormation VPC endpoints** + **S3 gateway endpoint**.
9. (a) **External / on-premises systems** signalling through the presigned URL. (b) **Receiving data** (`GetAtt .Data`). Also waiting at a specific point with `DependsOn`.
10. `cfn-signal -d "<data>"` to a WaitCondition handle, output `!GetAtt WaitCondition.Data`.
11. `AutoScalingReplacingUpdate WillReplace: true`, plus a **CreationPolicy** and cfn-signal to judge the new ASG.
12. `SuspendProcesses` (HealthCheck, ReplaceUnhealthy, AZRebalance, AlarmNotification, ScheduledActions).
13. `AutoScalingScheduledAction: IgnoreUnmodifiedGroupSizeProperties: true`; don't hard-code `DesiredCapacity`.
14. **No UpdatePolicy**: launch template changes only affect new instances.
15. `AutoScalingReplacingUpdate` (WillReplace takes precedence).

Quick memory summary

- **cfn-init** reads `AWS::CloudFormation::Init` (configSets; **packages, groups, users, sources, files, commands, services**); private S3 via **Authentication**; needs a route to CloudFormation; logs `/var/log/cfn-init*.log`.
- **cfn-signal** `-e $?` to the resource with the **CreationPolicy** (ASG / instance); signal after a health check.
- **CreationPolicy:** Count, Timeout (**max 12 h**), MinSuccessfulInstancesPercent; failure or timeout = rollback.
- **WaitCondition + handle:** presigned S3 URL, external systems, `GetAtt .Data`, doesn't re-wait on update.
- **cfn-hup:** daemon, `cfn-hup.conf` + `hooks.d` (`post.update`, rerun cfn-init); metadata changes in place. UserData = first boot only.
- **UpdatePolicy: RollingUpdate** (MaxBatchSize, MinInstancesInService, WaitOnResourceSignals, PauseTime, SuspendProcesses) vs **ReplacingUpdate** (WillReplace, new ASG, fast rollback, wins if both) + **ScheduledAction** (IgnoreUnmodifiedGroupSizeProperties). No UpdatePolicy = existing instances not replaced.